

DWM_Experiment no_06

AIM: Implementation of Prediction Algorithm (Linear Regression)

✓ Introduction:

Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.

Procedure :

1. Import Necessary Libraries: Begin by importing essential libraries such as NumPy, Pandas, Matplotlib, and scikit-learn.
2. Load and Explore the Dataset: Load your dataset using Pandas and perform exploratory data analysis to understand the data structure and relationships.
3. Preprocess the Data: Handle missing values, encode categorical variables if any, and split the dataset into independent (features) and dependent (target) variables.
4. Split the Dataset: Divide the data into training and testing sets, typically using an 80-20 split, to evaluate the model's performance on unseen data.
5. Train the Linear Regression Model: Use scikit-learn's LinearRegression class to fit the model on the training data.
6. Make Predictions: Utilize the trained model to predict outcomes on the test set.
7. Evaluate the Model: Assess the model's performance using evaluation metrics such as Mean Squared Error (MSE) and R-squared (R^2) score.
8. Visualize the Results: Plot the regression line along with the data points to visualize the fit and understand the model's predictions.

✓ CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sample dataset
data = {
    'Experience': [1, 2, 3, 4, 5],
```

```

    'Salary': [45000, 50000, 60000, 80000, 110000]
}
df = pd.DataFrame(data)

# Display the dataset
print(df)

```

```

↗
  Experience  Salary
0          1   45000
1          2   50000
2          3   60000
3          4   80000
4          5  110000

```

```

# Define features and target variable
X = df[['Experience']] # Independent variable
y = df['Salary']       # Dependent variable

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a linear regression model
model = LinearRegression()

# Train the model using the training data
model.fit(X_train, y_train)

```

```

↗
  LinearRegression ⓘ ?
  LinearRegression()

```

```

# Predict salaries for the test set
y_pred = model.predict(X_test)

# Calculate Mean Squared Error and R-squared
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")

```

```

↗
Mean Squared Error: 18367346.938775547
R-squared: nan
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_regression.py:1266: UndefinedMetricWarning:
  warnings.warn(msg, UndefinedMetricWarning)

```

```

# Plotting the regression line
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Experience vs. Salary')
plt.legend()
plt.show()

```



✓ Review Questions

1. What are the key steps involved in implementing a simple linear regression model using Python and scikit-learn?

1. Data Collection and Preparation

Collect Data: Gather a dataset containing the independent variable (predictor) and the dependent variable (response).

Preprocess Data: Handle missing values, encode categorical variables if necessary, and ensure the data is in the correct format for analysis.

2. Splitting the Dataset Training and Testing Sets: Divide the dataset into a training set to train the model and a testing set to evaluate its performance. A common split is 80% for training and 20% for testing.

3. Model Initialization Instantiate the Model: Create an instance of the LinearRegression class from scikit-learn, which will be used to fit the linear model to the data.

4. Model Training Fit the Model: Use the training data to fit the linear regression model. The model will learn the coefficients (slope and intercept) that define the best-fitting line through the data points.

5. Making Predictions Predict Outcomes: Apply the trained model to the testing data to predict the dependent variable's values based on the independent variable(s).

6. Model Evaluation Assess Performance: Evaluate the model's accuracy and performance using metrics such as Mean Squared Error (MSE) and the coefficient of determination (R^2 score). These metrics help determine how well the model captures the relationship between variables.

7. Visualization (Optional) Plot Results: Visualize the data points and the regression line to interpret the model's fit and the relationship between variables.

2. How can you evaluate the performance of a linear regression model in Python? List and explain at least two metrics.

1. Mean Squared Error (MSE)

Definition: MSE measures the average squared difference between the predicted values and the actual values.

Interpretation: A lower MSE indicates that the model's predictions are closer to the actual values, signifying better performance.

2. R-squared (Coefficient of Determination)

Definition: R-squared represents the proportion of the variance in the dependent variable that is predictable from the independent variables.

Interpretation: An R-squared value closer to 1 indicates that a large proportion of the variance in the dependent variable has been accounted for by the model.

3. What is the role of the `train_test_split()` function in building a linear regression model, and why is it important?

The `train_test_split()` function in scikit-learn is essential for evaluating a linear regression model's performance.

Purpose:

It divides your dataset into two subsets:

Training Set: Used to train the model.

Testing Set: Used to evaluate the model's performance on unseen data.

This separation ensures that the model is tested on data it hasn't seen during training, providing a more accurate assessment of its generalization ability.

Importance:

Prevents Overfitting: By training and testing on separate data, the model is less likely to memorize the training data, leading to better generalization.

Simulates Real-World Performance: It mimics how the model will perform on new, unseen data, which is the ultimate goal in predictive modeling.

Enables Model Comparison: Using consistent training and testing splits allows for fair comparison between different models or algorithms.

✓ **CONCLUSION:**

In this implementation, we utilized linear regression to model and predict continuous outcomes based on one or more independent variables. Linear regression serves as a foundational technique in data science, offering a straightforward approach to understanding relationships between variables and making predictions.

GITBUB: <https://github.com/panchaldeep1123/dwm>